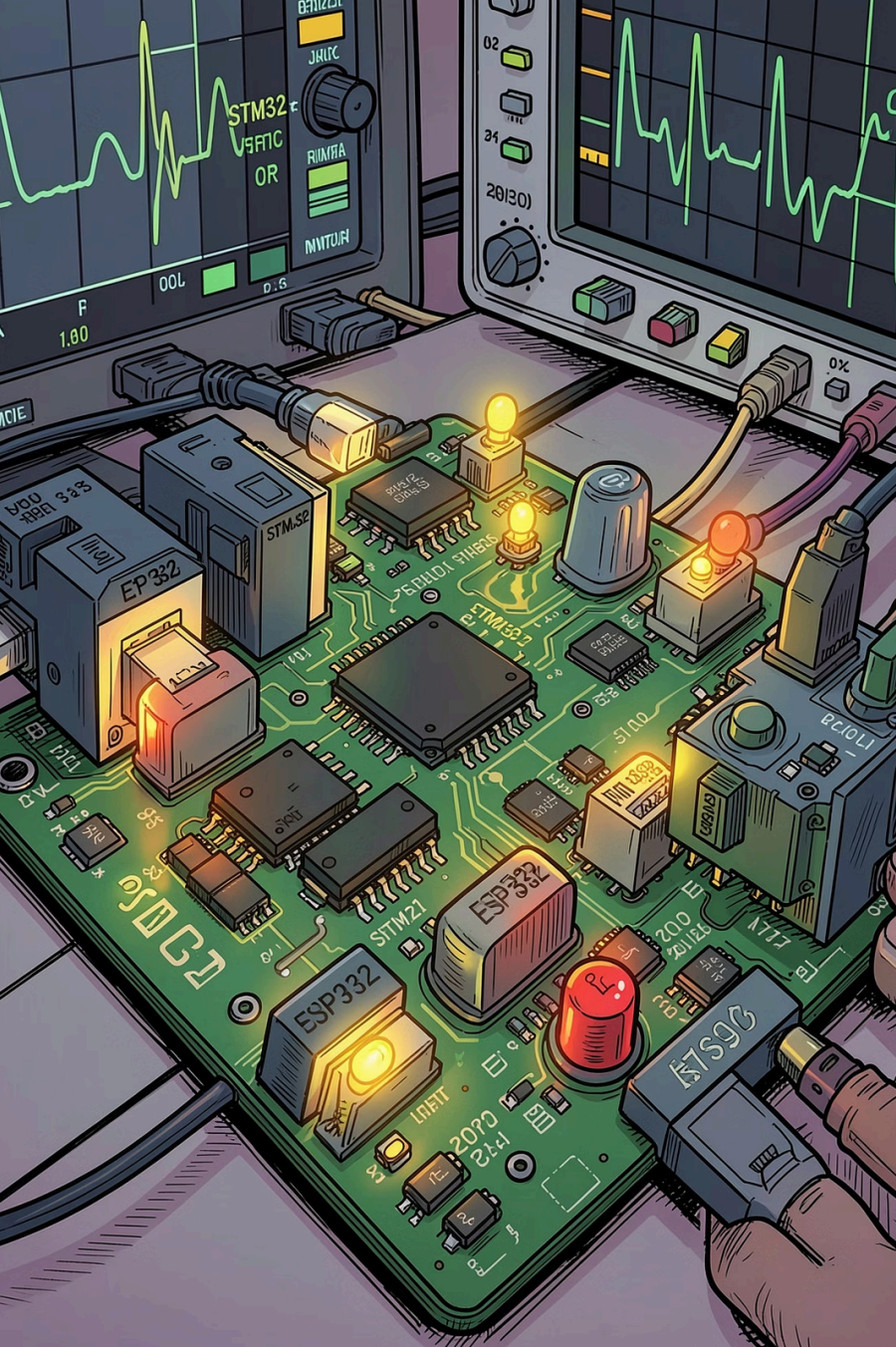




Modul FreeRTOS Middle

Pengembangan sistem embedded dengan RTOS menuju penyelesaian **25 eksperimen final** — 10 STM32, 10 ESP32, dan 5 Multi MCU — serta project akhir **RTOS Sensor Hub System** yang mengintegrasikan GPIO, interrupt, encoder, UART, DAC, ADC, I2C, dan SPI dalam satu sistem real-time terpadu.

EMBEDDED SYSTEMS

FREERTOS

STM32 · ESP32

Dasar FreeRTOS

Konsep Inti

FreeRTOS adalah kernel RTOS open-source untuk mikrokontroler dengan **multitasking preemptive**. Task adalah unit eksekusi mandiri — fungsi dengan loop tak terbatas dan delay non-blocking. Scheduler mengatur eksekusi berdasarkan prioritas; context switch menyimpan dan memulihkan state task secara otomatis.

- Task: fungsi dengan parameter `void*`, loop `for(;;)`
- Scheduler: prioritas 0 (idle) hingga `configMAX_PRIORITIES-1`
- Context switch: simpan register & stack, pulihkan state task berikutnya

RTOS Primitives

FreeRTOS menyediakan mekanisme komunikasi dan sinkronisasi antar-task:

Semaphore

Binary & counting untuk signaling ISR→task

Mutex

Mutual exclusion dengan priority inheritance

Queue

Data transfer thread-safe antar-task/ISR

Event Group

Sinkronisasi berbasis bitmask multi-event

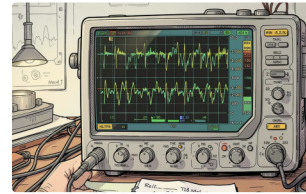
- 📄 Praktikum Modul 09: minimal **3 task** dan **2 RTOS primitives** per eksperimen.

Task Creation dan Scheduler



Membuat Task

ESP32: `xTaskCreate()` · STM32 CMSIS-RTOS: `osThreadNew()`. Parameter: fungsi task, nama, stack depth (words), parameter `void*`, prioritas, dan task handle. Stack depth harus cukup untuk variabel lokal, function call, dan context save — stack kurang menyebabkan **stack overflow**.



Menjalankan Scheduler

ESP32: scheduler berjalan otomatis setelah `app_main()` selesai. STM32Cube: panggil `osKernelStart()` atau `vTaskStartScheduler()` secara eksplisit. Task delay: `vTaskDelay()` (tick-based) atau `vTaskDelayUntil()` (fixed frequency).

Hindari

ISR dan RTOS

Aturan ISR di FreeRTOS

ISR harus **sangat cepat dan non-blocking** karena berjalan di context interrupt dengan stack terpisah. Gunakan hanya API berakhiran `FromISR()`:

→ `xSemaphoreGiveFromISR()`

Berikan semaphore dari ISR ke task

→ `xQueueSendFromISR()`

Kirim data ke queue dari ISR

→ `xEventGroupSetBitsFromISR()`

Set bit event group dari ISR

⚠ **Jangan gunakan** `vTaskDelay()` atau `xQueueSend()` (tanpa `FromISR()`) di dalam ISR — akan menyebabkan error atau crash.

Pola: ISR → Semaphore → Task

Contoh EXTI button: ISR hanya memanggil `xSemaphoreGiveFromISR()` dan `portYIELD_FROM_ISR()` jika ada task prioritas lebih tinggi yang unblocked. Task menunggu dengan `xSemaphoreTake()` dan menangani logika button termasuk debounce.

- ISR: minimal, hanya signaling
- Task: semua pemrosesan logika
- Debounce: di task level, bukan di ISR
- `xHigherPriorityTaskWoken`: flag untuk context switch setelah ISR

Semaphore dan Mutex

Binary Semaphore

Signaling satu-ke-satu, ideal untuk ISR → task. ISR memanggil

`xSemaphoreGiveFromISR()`, task menunggu dengan `xSemaphoreTake()`.

Tidak dimiliki oleh task tertentu.

Counting Semaphore

Resource pool dengan jumlah terbatas.

Counter bertambah saat resource tersedia, berkurang saat diambil. Contoh: pool buffer DMA atau slot memori.

Mutex

Proteksi shared resource dengan **mutual exclusion**. Dimiliki oleh task yang mengambilnya dan harus dilepas oleh task yang sama. Mendukung **priority inheritance** untuk mencegah priority inversion.

Contoh Semaphore

ISR button → `xSemaphoreGiveFromISR()` → task menunggu → toggle LED. ISR tetap cepat, logika di task level.

Priority Inheritance

Jika task prioritas rendah memegang mutex dan task prioritas tinggi menunggu, prioritas task rendah **dinaikkan sementara** agar segera melepas mutex — mencegah starvation pada task prioritas tinggi.

Queue dan Event Group

Queue — Inter-Task Data Transfer

Queue adalah mekanisme komunikasi thread-safe untuk data bertipe bebas (struct, int, char). Bersifat **FIFO**, dibuat dengan ukuran dan tipe data tertentu. Dapat diakses dari multiple task dan ISR tanpa race condition.

`xQueueSend()`

Kirim dari task (dengan timeout)

`xQueueSendFromISR()`
)

Kirim dari ISR (ISR-safe)

`xQueueReceive()`

Terima dari queue (dengan timeout)

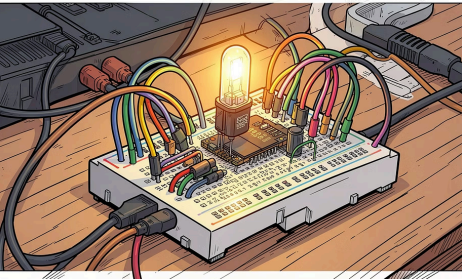
Contoh: ADC task → kirim hasil ke queue → display task terima dan tampilkan. ISR UART RX → queue → task processor command.

Event Group — Multi-Event Sync

Event group terdiri dari **8 atau 24 bit** (tergantung konfigurasi) yang dapat diset/clear secara individual. Task menunggu satu atau beberapa bit dengan `xEventGroupWaitBits()` — tunggu **any bit** atau **all bits**.

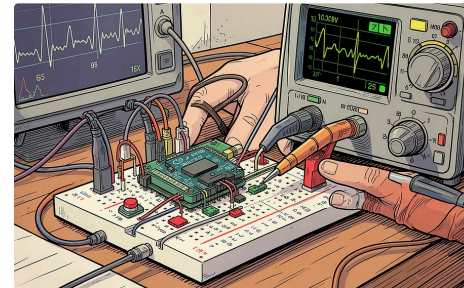
- Task display tunggu: ADC ready **ATAU** sensor I2C ready **ATAU** command UART
- ISR set bit dengan `xEventGroupSetBitsFromISR()`
- Sinkronisasi start: semua task tunggu bit START, lalu jalan bersamaan
- Task tahu event mana yang terjadi setelah unblocked

GPIO Tasks dan External Interrupt



GPIO Task — Blink LED

ESP32: `gpio_pad_select_gpio()`, `gpio_set_direction()`, `gpio_set_level()`. Task blink: `vTaskDelay(pdMS_TO_TICKS(500))`. STM32: `HAL_GPIO_TogglePin()` dengan `osDelay(500)`. **Penting:** gunakan `vTaskDelay()/osDelay()`, bukan `HAL_Delay()` yang blocking.



External Interrupt — Button

STM32: `HAL_GPIO_EXTI_IRQHandler()` → `xSemaphoreGiveFromISR()`. ESP32: `gpio_install_isr_service()`, `gpio_isr_handler_add()` → `xSemaphoreGiveFromISR()`. Task menunggu semaphore, menangani aksi (toggle LED, counter). **Debounce di task level**, bukan di ISR.

i Beberapa task GPIO dapat berjalan bersamaan dengan prioritas dan delay berbeda — semua dikelola scheduler RTOS secara preemptive.

Encoder 2-Pin dan Serial UART

Rotary Encoder 2-Pin (Quadrature)

Encoder memiliki pin A dan B dengan phase 90°. Interrupt pada pin A: baca pin B untuk tentukan arah.

- B = HIGH saat A falling → **CW** (ClockWise)
- B = LOW saat A falling → **CCW** (Counter ClockWise)

ISR kirim arah ke **queue** (1 = CW, -1 = CCW). Task encoder proses queue, hitung counter, update display. Debounce: ignore event dalam jendela 5 ms atau filter state machine.

UART Serial dengan RTOS

ESP32: `uart_driver_install()` dengan RX/TX buffer dan queue event. ISR UART RX → queue → task RX baca dengan `uart_read_bytes()`. Task TX kirim dengan `uart_write_bytes()`.

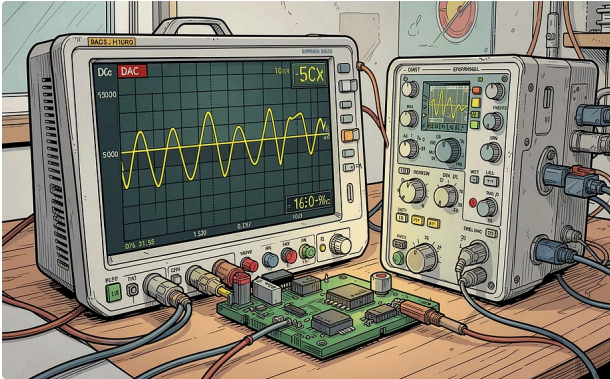
STM32: `HAL_UART_Receive_IT()` → callback `HAL_UART_RxCpltCallback()` → `xQueueSendFromISR()`. Task processor baca queue dan eksekusi command.

LED ON/OFF

ADC START/STOP

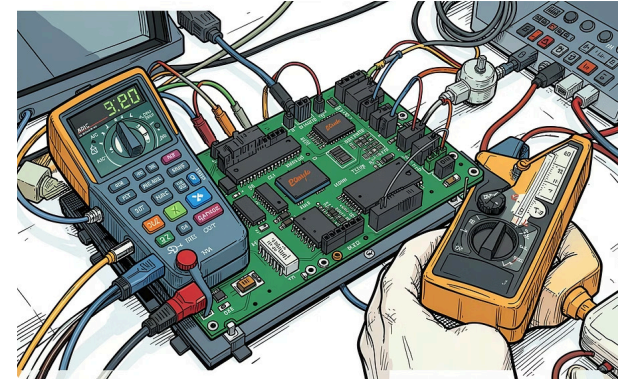
GET STATUS

DAC dan ADC dengan RTOS



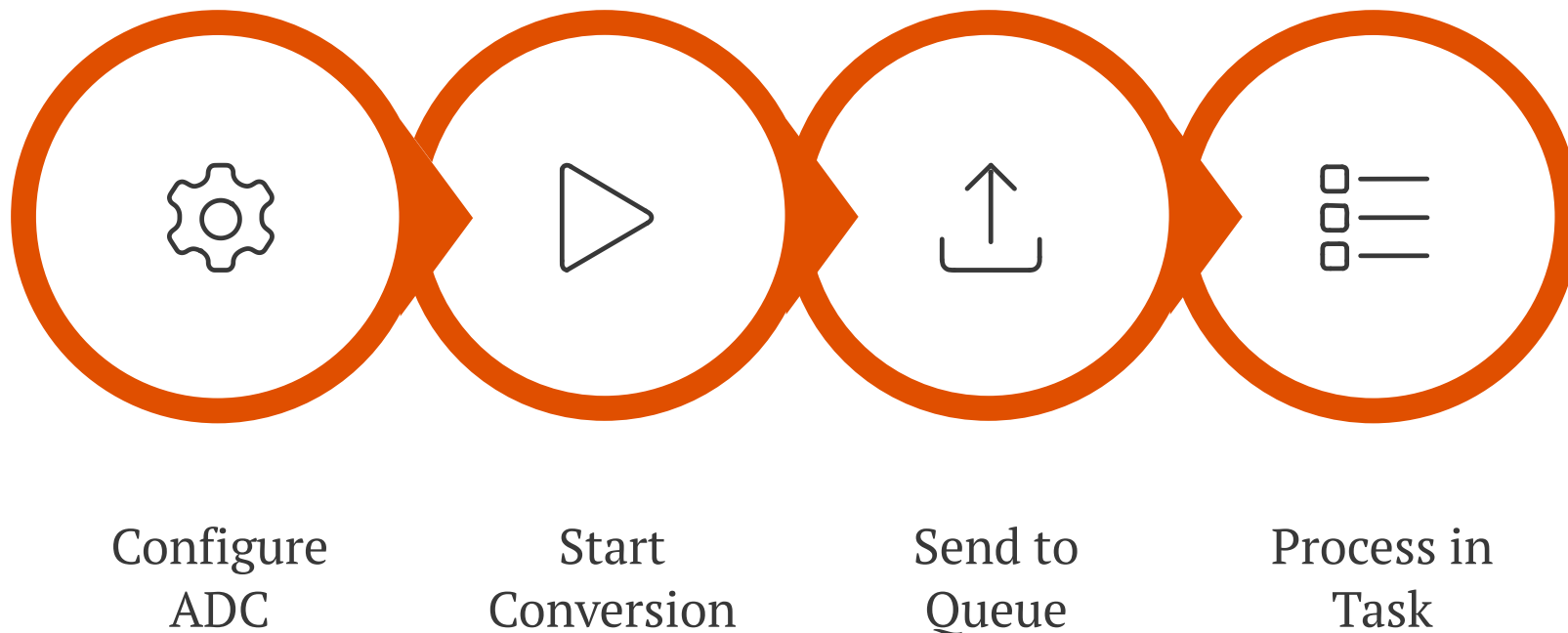
DAC — Generate Sinyal Analog

ESP32: `dac_output_enable()`, `dac_output_voltage()` (8-bit, 0–255 → 0–3.3V). Lookup table untuk sin, triangle, sawtooth. Delay task kontrol frekuensi: `vTaskDelay(pdMS_TO_TICKS(1)) + 100 sampel/cycle` → 1 kHz. STM32: `HAL_DAC_Init()`, `HAL_DAC_SetValue()` (8/12-bit). Osiloskop verifikasi bentuk gelombang dan frekuensi output.



ADC — Baca Tegangan Analog

ESP32 ADC1: `adc1_config_width(ADC_WIDTH_12BIT)`, `adc1_config_channel_atten()` (0–11dB untuk range 0–3.3V). Baca: `adc1_get_raw()` → konversi: $\text{voltage} = \text{adc_val} \times 3.3 / 4095$. STM32: `HAL_ADC_Start()`, `HAL_ADC_GetValue()`. Hasil dikirim ke queue untuk display/UART. Moving average filter di task untuk menghaluskan fluktuasi.



Alur pembacaan ADC dalam lingkungan RTOS memastikan konversi analog-to-digital berjalan periodik tanpa memblokir task lain, dengan hasil yang dikirim melalui queue untuk diproses lebih lanjut.

I2C, SPI, dan Target Akhir

I2C dengan Mutex

I2C bus shared harus dilindungi mutex. ESP32: `i2c_driver_install()` → `task xSemaphoreTake()` → `i2c_master_write_read_device()` → `xSemaphoreGive()`. STM32: `HAL_I2C_Mem_Read/Write()`. Mutex mencegah race condition saat multiple task (sensor, display, logger) mengakses bus yang sama.

SPI dengan Mutex

ESP32: `spi_bus_initialize()`, `spi_device_transmit()`. STM32: `HAL_SPI_TransmitReceive()` atau DMA. Aplikasi: SD card, sensor ADXL345, display, komunikasi antar-MCU. Transfer non-blocking dengan callback/semaphore notifikasi selesai.

Project Akhir: RTOS Sensor Hub System

Integrasi semua komponen dalam satu sistem real-time:

01

GPIO Tasks

LED, output kontrol periodik

02

Interrupt & Encoder

EXTI button, rotary encoder 2-pin

03

Serial UART

Command protocol, RX/TX queue

04

DAC & ADC

Waveform generation, sensor reading

05

I2C & SPI

Sensor read, data logging, display

10

Eksperimen STM32

FreeRTOS via CMSIS-RTOS / STM32Cube

5

Eksperimen Multi MCU

Komunikasi STM32 ↔ ESP32

10

Eksperimen ESP32

FreeRTOS native ESP32

25

Total Eksperimen

Menuju project akhir terpadu